Monitoring Workflow Framework (MLflow-Monitor)

Project Proposal

Date: 2026-03-09

This proposal describes a practitioner-first framework for offline, post-deployment monitoring runs. It standardizes baseline selection, comparability, evidence capture, and actionable diffs, while supporting templates and user-defined recipes. MLflow is treated as an optional sink/viewer, not the center of the product.

1. Where the idea originated

The idea originated from hands-on experience in small ML teams where drift and post-deployment monitoring are acknowledged as important but remain difficult to operationalize. In particular, at Oden you observed that MLflow often serves as the “center of gravity” for experiment tracking, yet practitioners still end up writing bespoke scripts and manually logging artifacts/metrics to approximate monitoring workflows. The recurring friction is not the absence of individual metrics, but the lack of a repeatable, baseline-aware, diff-first monitoring workflow that is easy to run on a schedule and easy to compare over time.

2. Fundamental problems distilled from the idea

Across teams, the deepest pain points cluster around workflow rather than algorithms:

bullet Baseline is not first-class: teams struggle to pin, manage, and reuse baselines (dataset snapshots, prior runs, model versions).

bullet Comparability is brittle: schema/feature changes and inconsistent logging make run-to-run comparisons error-prone.

bullet Results are not action-oriented: raw metrics create noise; practitioners need ranked deltas, severity, and a short action list.

bullet Evidence is missing: when something looks wrong, teams cannot quickly answer what code, data, and environment produced the result.

bullet Integration is awkward: practitioners repeatedly build ad-hoc glue to store results in MLflow or other systems.

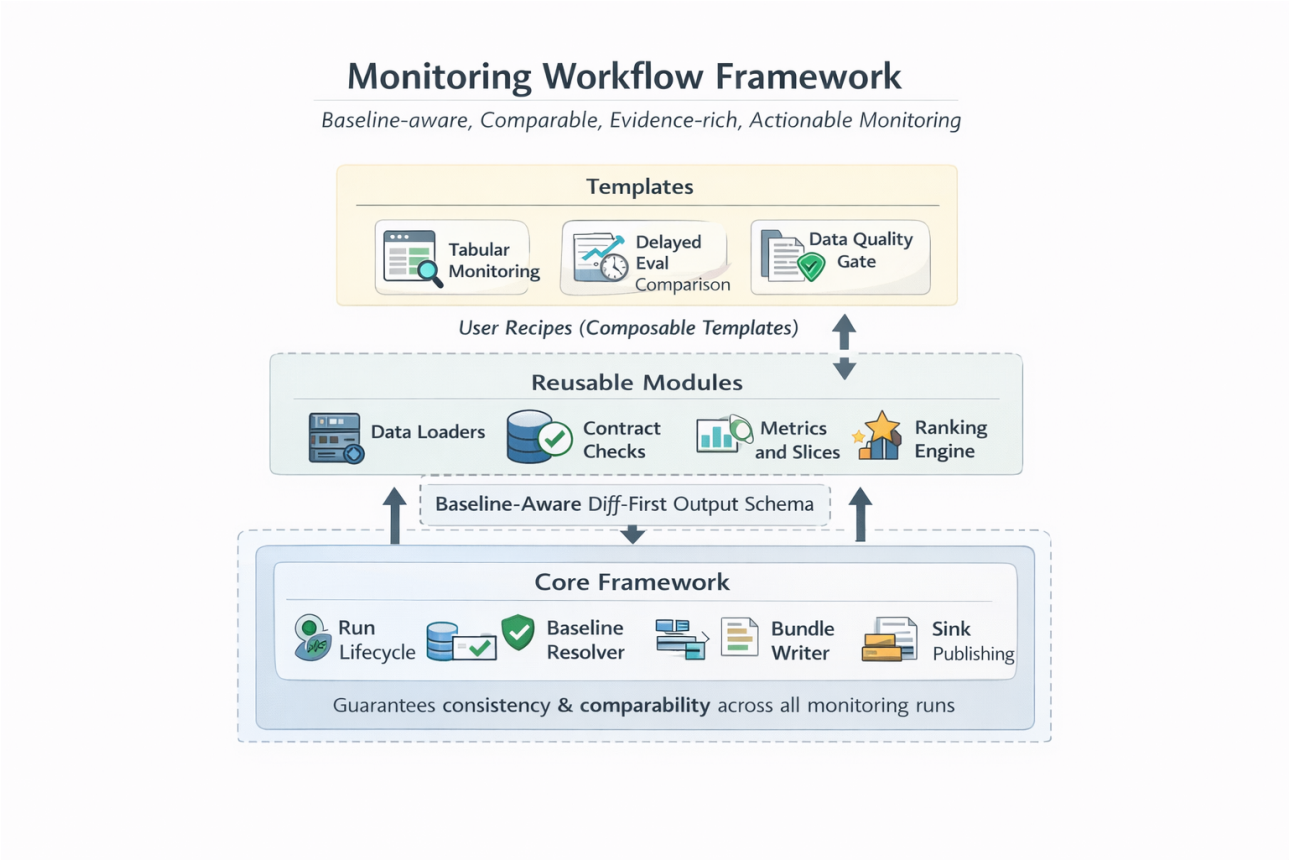
3. Concept of the proposed solution

Build a foundational “Monitoring Workflow Framework” that guarantees baseline-aware, comparable, evidence-rich, actionable monitoring runs. The framework provides the plumbing once (run lifecycle, baseline resolver, standardized output bundle, diff engine, evidence capture, and sinks). On top of it, users run templates or compose their own recipes from reusable modules without rewriting baseline/diff/bundling logic.

- Core Framework: run lifecycle, baseline resolver + compatibility checks, bundle writer, sink publishing (local first; MLflow optional).
- Reusable Modules: data loaders, contract checks, metrics/slice engine, ranking/severity engine, report renderer.
- Templates/Recipes: opinionated assemblies for common workflows; user customization is primarily composition/configuration, not boilerplate.

4. High-level summary

You are building a foundational framework for offline, post-deployment monitoring runs that makes monitoring baseline-aware, comparable, evidence-rich, and actionable—regardless of the specific monitoring use case. The core standardizes the workflow (baseline → validate/contract → compute checks → diff + rank → report + evidence → publish/history), while reusable modules and templates provide out-of-the-box value and a path for practitioners to encode their own workflows. MLflow is an optional sink/viewer for run history and artifacts, not the center of the product.

5. System diagram

Three-layer architecture: Core Framework → Reusable Modules → Templates/User Recipes.